

Documentación del módulo de esperas de eShop Core

Instituto Superior de Formación Técnica N. º 179 Dr. Carlos Pellegrini

Carrera: Tecnicatura Superior en Análisis de Sistemas

Materia: Algoritmos y Estructura de Datos III

Docente: Hansen Andres

Ayudante de cátedra: Rios Emmanuel

Tema: Desarrollo de aplicación con TAD

Estudiantes: Trujillo Sebastian y Sanoguera Agustin

El módulo BACKEND/esperas.py gestiona a los clientes que solicitan productos sin disponibilidad. Su función es conservar el orden de llegada, recuperar las reservas guardadas y colaborar con el inventario para atender al primer cliente cuando se repone stock. La reserva pertenece al cliente identificado en el portal, sin asignación manual desde la pantalla de inventario.

Esta documentación explica el funcionamiento de las seis funciones del módulo y su relación con el servidor, el TDA Cola, la persistencia, el inventario y la interfaz. El análisis se basa en el código local leído el 28 de octubre de 2026. Los ejemplos son explicativos y no implican ejecución del sistema ni inserción de datos de prueba.

Funcionamiento del módulo

Organización de las colas y de sus registros

El estado se representa mediante un diccionario llamado colas. Cada clave es un id_producto y su valor es una instancia del TDA Cola de la cátedra. El diccionario permite localizar la cola correspondiente; los clientes en espera se almacenan en los nodos de esa estructura. Dos productos tienen colas independientes.

Se utiliza la política FIFO, primero en entrar y primero en salir. Una reserva nueva ingresa por el final con arribo, mientras que la atención se realiza por el frente con atencion. El módulo utiliza primitivas y no modifica directamente los enlaces frente, final o sig.

La Tabla 1 presenta campos de cada registro de espera.

Tabla 1

Campos de cada registro de espera

Campo	Contenido	Origen
id_espera	Identificador del registro y referencia de su orden de llegada.	SQLite, al insertar la fila.
id_cliente	Identificador del cliente que reserva.	Sesión de ingreso resuelta por el servidor.
id_producto	Identificador del producto agotado.	Producto seleccionado en el catálogo.
fecha_espera	Fecha y hora en formato ISO 8601, con zona horaria UTC.	Backend, al registrar la espera.

Nota. Elaboración a partir del código local de eShop Core. Los identificadores de funciones se conservan como aparecen en Python.

La fecha se genera con datetime.now(timezone.utc).isoformat(timespec="seconds"). El módulo datetime permite representar fechas con información de zona horaria. En el sistema, la prioridad se determina por id_espera ascendente, no por comparar el texto de la fecha.

Carga inicial de las colas

cargar_colas() se ejecuta al iniciar el servidor. Solicita todos los registros a persistencia.leer_esperas(), que los entrega ordenados por id_espera. Por cada registro localiza su producto, crea una Cola si aún no existe y utiliza arribo para incorporar una copia del diccionario.

La función devuelve el diccionario de colas reconstruido. Si no existen reservas, devuelve {}. No inserta registros en SQLite ni crea colas para productos que todavía no tienen esperas.

Aunque los IDs se asignen globalmente, su orden conserva la secuencia de llegada dentro de cada producto.

Consulta de todas las esperas de un producto

consultar_esperas(colas, id_producto) devuelve una lista de copias de los registros en orden FIFO. Si la cola no existe o está vacía, devuelve []. La lista es un resultado para consulta; la estructura que gestiona la espera sigue siendo el TDA Cola.

Para recorrer la cola, obtiene su tamaño inicial y realiza exactamente esa cantidad de movimientos con mover_al_final. Cada movimiento retira el primero y lo agrega al final. Al completar una vuelta, la secuencia queda como estaba al comenzar. Por ejemplo, A, B, C pasa temporalmente por B, C, A y C, A, B y vuelve a A, B, C.

La consulta cuesta O(n) para n esperas y genera O(n) registros de salida. Como el recorrido cambia temporalmente la posición de los elementos, el servidor debe realizarlo bajo el mismo bloqueo utilizado para registrar y atender clientes.

Consulta de la primera espera

consultar_primera_espera(colas, id_producto) comprueba si existe una cola con elementos. Si no hay nadie esperando, devuelve None. Si hay registros, usa en_frente y devuelve una copia del primero, sin retirarlo ni escribir en la base. Esta operación sobre el TDA es O(1).

Registro de una nueva espera

registrar_espera(colas, id_cliente, id_producto) exige identificadores enteros positivos. La comparación estricta type(valor) is int evita aceptar True o False como IDs. Luego genera la fecha y llama a persistencia.insertar_espera para guardar el registro.

La persistencia comprueba que el cliente y el producto existan, que el stock sea cero y que no haya otra espera activa del mismo cliente para ese producto. Si el guardado falla o se rechaza, la cola queda intacta. Si se confirma, se crea la cola cuando es necesario, se incorpora la espera con arribo y se devuelve una copia del registro creado.

Un cliente puede esperar distintos productos y varios clientes pueden esperar el mismo artículo. No se registran cantidades reservadas en esta operación. La espera no modifica stock, no confirma una compra y no cambia la popularidad.

Preparación de la notificación

preparar_notificacion(colas, id_producto, nombre_producto) consulta al primero de la cola y obtiene los datos de ese cliente mediante persistencia.consultar_cliente. Si la cola está vacía, devuelve None. Si el cliente referenciado no existe, lanza ValueError.

La respuesta contiene id_espera, id_cliente, id_producto y un mensaje de notificación simulada con nombre del cliente y del producto. Esta función prepara información: todavía no elimina la espera, no guarda el aviso y no envía mensajes externos. El aviso persistente se genera después, dentro de la operación SQL coordinada por el inventario.

Confirmación de la atención

confirmar_atencion(colas, id_producto) ejecuta atencion sobre la cola del producto y devuelve una copia del registro retirado. Se llama únicamente después de confirmar en SQLite la reposición y la atención. No decide por sí misma si corresponde atender ni valida nuevamente el stock.

La función presupone que la cola existe y contiene al cliente previamente consultado. Por eso la preparación, el guardado y el retiro deben permanecer coordinados por el bloqueo

compartido del servidor. Usar atencion sobre una cola vacía incumpliría la precondición de esta primitiva.

Relación con los demás módulos

La Tabla 2 presenta relaciones del módulo de esperas con el sistema.

Tabla 2

Relaciones del módulo de esperas con el sistema

Módulo	Relación con esperas.py
BACKEND/tad_cola.py	Provee Cola, arribo, atencion, cola_vacia, en_frente, tamanio y mover_al_final.
BD/persistencia.py	Guarda y recupera esperas, consulta clientes y confirma operaciones SQL. esperas.py lo importa directamente.
BACKEND/servidor.py	Carga las colas, identifica al cliente, atiende rutas HTTP y comparte el Lock entre consultas, reservas y reposiciones.
BACKEND/inventario.py	Detecta la transición de stock cero a positivo y llama a preparar_notificacion y confirmar_atencion.
FRONTEND/portal.js	Envía reservas del cliente y presenta turnos y avisos recibidos desde la API.
BACKEND/transacciones.py	Las compras ordinarias no retiran esperas. La función interna de reversión puede atenderlas al restituir stock, pero no está expuesta por la API actual.

Nota. Elaboración a partir del código local de eShop Core. Los identificadores de funciones se conservan como aparecen en Python.

Relación con el servidor y la identificación del cliente

Al iniciar el sistema, servidor.py llama a cargar_colas una vez y entrega ese estado a todas las solicitudes. Al reservar, el servidor recibe únicamente id_producto y la cabecera X-Sesion. cliente_actual resuelve el token y obtiene el ID del cliente que ingresó por DNI. El navegador no elige otro cliente para asociarlo a la espera.

La ruta GET /api/mis-reservas recorre las colas, calcula el turno de cada registro y filtra los correspondientes al cliente actual. Para hacerlo utiliza consultar_esperas, conservando el orden final. La ruta GET /api/productos/ID/esperas consulta la cola completa de un producto existente; si el producto no existe responde 404.

Relación con la persistencia y sus transacciones

La tabla esperas guarda los cuatro campos del registro y tiene claves foráneas hacia clientes y productos. leer_esperas puede recuperar todas las filas para reconstruir el estado o filtrar por producto. insertar_espera mantiene sus comprobaciones y la inserción dentro de BEGIN IMMEDIATE.

Durante la reposición, persistencia.modificar_producto verifica que la espera indicada sea la primera guardada para el producto. En una misma transacción actualiza las existencias, llama a guardar_aviso y elimina la espera. Así, un fallo impide confirmar solo una parte de esa operación. El administrador de contexto de la conexión permite confirmar o revertir la transacción según su resultado.

La función eliminar_espera existe como operación independiente de persistencia, pero el flujo normal de reposición utiliza la eliminación integrada en modificar_producto. El aviso queda en notificaciones y el cliente puede recuperarlo aunque cierre la página.

Relación con inventario y reposición

El módulo inventario compara la cantidad anterior con la nueva. Solo cuando el producto pasa de cero a un valor positivo prepara la atención del primero. Después de confirmar la operación SQL, reemplaza el producto dentro de la lista enlazada y llama a confirmar_atencion para retirar el registro de la cola.

Si no hay nadie esperando, se modifica el stock sin retirar registros ni crear un aviso. Si el stock pasa de cinco a diez, no se activa una nueva atención: la condición implementada es la transición desde cero. Reponer varias unidades atiende a un cliente, no a toda la cola.

Relación con el portal del cliente

Cuando el catálogo muestra un producto agotado, aparece Reservar. portal.js envía POST /api/esperas y vuelve a consultar los datos. El carrito presenta las reservas en En espera de stock, con nombre del producto, turno e identificador. Si la espera ya está registrada, el catálogo muestra ese estado e impide repetir la acción desde el botón.

Los avisos de reposición se consultan mediante GET /api/mis-notificaciones y se muestran en Avisos de reposición. La notificación es simulada dentro del sistema; no envía correo ni aparta unidades. Vaciar los productos del carrito no elimina las esperas persistentes.

Flujos completos de funcionamiento

Desde la reserva hasta la cola

1. El cliente ingresa por DNI y selecciona Reservar en un producto con stock cero.
2. El servidor valida JSON, sesión e identificador del producto y entra en el bloqueo compartido.
3. registrar_espera genera la fecha y solicita a SQLite la inserción validada.
4. Tras confirmar el registro, arribo lo incorpora al final de la cola correspondiente.
5. La API devuelve 201 y el portal actualiza la sección En espera de stock.

Desde la reposición hasta el aviso

1. El inventario recibe una modificación que cambia el stock de cero a positivo.
2. preparar_notificacion consulta al primero y obtiene los datos del cliente, sin retirarlo todavía.

3. SQLite confirma juntos stock, aviso persistente y eliminación de la espera.
4. Se actualiza el inventario en memoria y confirmar_atencion retira al primero de la cola.
5. El cliente recupera el aviso desde su carrito y puede efectuar una compra por separado.

Recuperación después de reiniciar

Al detener el proceso se pierde la representación en memoria, pero no las filas de esperas ni las notificaciones. En el próximo inicio, cargar_colas reconstruye el orden desde SQLite. El cliente vuelve a identificarse por DNI y el servidor filtra las reservas que siguen activas para su ID.

Ejemplo de atención FIFO

Supóngase un producto con stock cero reservado, en ese orden, por los clientes A, B y C. Se trata de un ejemplo conceptual: no se crean registros en la base para explicarlo.

La Tabla 3 presenta evolución de una cola al reservar y reponer.

Tabla 3

Evolución de una cola al reservar y reponer

Evento	Cola después del evento	Stock
Reserva A	A	0
Reserva B	A, B	0
Reserva C	A, B, C	0
Se reponen diez unidades	B, C; A recibe el aviso y sale de la espera.	10
A compra dos unidades	B, C; la compra se procesa por separado.	8

Nota. Elaboración a partir del código local de eShop Core. Los identificadores de funciones se conservan como aparecen en Python.

El ejemplo muestra que la espera no es una venta ni una asignación automática de unidades. El aviso informa disponibilidad. La compra posterior descuenta stock mediante el

módulo de transacciones y actualiza la popularidad; la cola de los demás clientes conserva su orden.

Consistencia de los datos y límites del módulo

Copias de los registros

Las consultas y el registro devuelven copias de diccionarios para que quien recibe el resultado no modifique accidentalmente el dato dentro del TDA. En Python, asignar un objeto a otra variable no crea una copia; las copias superficiales independizan el diccionario externo.

En una espera los campos son enteros y texto, por lo que .copy() resulta apropiado. No crea otra fila de SQLite ni un respaldo en disco. Su función es proteger el estado en memoria que se entrega a otros módulos.

Bloqueo y errores

El Lock del servidor coordina el estado compartido del proceso y las transacciones SQLite coordinan los cambios duraderos. Son responsabilidades diferentes. El módulo esperas no crea su propio bloqueo: el servidor debe mantenerlo durante la reserva, las consultas que rotan la cola y la reposición con atención.

Si el cliente o el producto no existen, el producto tiene stock o la espera está duplicada, la persistencia rechaza el registro antes de agregarlo al TDA. Si falla el guardado de la reposición y la atención, no se confirma esa modificación ni se retira al cliente de la cola. La interfaz recibe un mensaje de error y puede volver a consultar el estado.